Trusted AI Safety Expert: Module 8

Official Study Guide

TAISE™
Trusted AI Safety Expert

CSA cloud security alliance®

Northeastern University

Version Number: 20251015

Acknowledgments

Lead Authors

Tugce Guler
Satwik Kamarthi
Wei Li
Charlie Meyer
Nihar Sanda
Sam Scarpino
James Sheldon
Thulasi Tholeti
Peter Vickers

Contributors

Cansu Canca
Shiran Dudy
Laura Haaber Ihle
Shantanu Jain
Tomo Lazovich
Resmi Ramachandranpillai
Annika Schoene

CSA and NU Global Staff

Marina Bregkou
Josh Buker
Daniele Catteddu
Holly Franklin
Ryan Gifford
Keagan Goodhue
Tara Hanson
Larry Hughes
Nicholas Knopf
Stephen Lumpe
NJ Park
Hannah Rock
Anna Schorr
Stephen Smith
Cathy Jo Swain
Kevin Tobin
Jamie Traynor
Kira Twombly

Table of Contents

Introduction to TAISE

If AI is going to shape the world, we need people who know how to shape it safely.

The **Trusted AI Safety Expert (TAISE)** certificate, created by the Cloud Security Alliance (CSA) and Northeastern University, is a rigorous, research-backed program for professionals who build, manage, or audit intelligent systems. Through 10 comprehensive modules and a final certificate exam, learners gain practical skills to evaluate and mitigate real-world AI risks, apply AI safety and security controls, navigate compliance frameworks, and lead responsible AI adoption across industries. TAISE is more than a certificate—**it's a commitment to advancing safe, secure, and responsible AI.**

Receive a Respected Industry Credential

TAISE is co-issued by CSA, **the global leader in cloud security standards**, and Northeastern University, **a top academic institution for AI ethics and experiential learning**. This joint recognition enhances your professional credibility, providing a globally respected credential to highlight on resumes, LinkedIn, and professional profiles.

Acquire In-Demand Skills

Professionals will learn to work with frameworks such as **NIST AI RMF, CSA AICM, ISO standards**, and **MITRE A TLAS** while mastering **threat classification, data privacy, model governance, and AI-specific incident response**. These capabilities directly map to the urgent needs of enterprises adopting AI.

Go Beyond Theory

The **TAISE Prompt Library** transforms concepts into hands-on practice. It offers curated prompts and exercises that show learners how to use large language models to reinforce and apply safety principles from each module. While not an exam prep tool, it helps professionals **translate training into job-ready skills.**

Module 8: Cloud & AI Security

This module explores the unique security considerations that emerge when deploying, managing, and operating AI systems in cloud environments. You will discover how traditional cloud security principles must evolve to address the specific vulnerabilities and threats that AI tools face, from model stealing and evasion attacks to the complexities of securing machine learning pipelines across distributed cloud infrastructure. Through examining the AWS Well-Architected Framework, Zero Trust Architecture tenets, and MLOps security practices, you will build the knowledge needed to design resilient, secure AI systems that can withstand both conventional cyber threats and AI-specific attack vectors. By the end of this module, you will be equipped with the knowledge underlying comprehensive security strategies that protect your organization's most valuable AI assets while also maintaining the scalability and flexibility that cloud platforms provide.

Please note that while this module uses AWS terminology and examples, security best practices are cloud-agnostic and also apply to Azure, GCP, and other providers.

Access the TAISE Prompt Library, a free collection of curated prompts designed to complement the TAISE training. This optional resource provides example prompts and techniques that show how to use large language models (LLMs) to explore, reinforce, and apply key ideas from each module.

Learning Objectives

In this module, you will learn to:

- Explain the fundamentals of cloud security as it applies to AI systems
- Implement secure deployment and management strategies on cloud platforms (IaaS, SaaS, and PaaS)
- Establish continuous monitoring practices and secure MLOps pipelines
- Develop incident response and disaster recovery plans for AI in cloud environments
- Apply Zero Trust Architecture principles to protect AI workloads

8.2 Module Vocabulary

Continuous Delivery (CD) refers to the practice of preparing integrated changes for release to production environments, ensuring software is always in a deployable state. It incorporates constant feedback from customers, validated by acceptance tests, to realize benefits faster and react quickly to change. By reducing deployment effort, it gives more time for feature development.

Continuous Integration (CI) refers to the practice of automatically and frequently integrating changes into shared resources. It focuses on blending the work of individual developers into a common repository. Each time a developer makes changes, the code is automatically built and tested, enabling faster detection of bugs. This provides the team with a more efficient feedback loop.

Data at Rest, meaning data stored in a database, file system, or object store, is a responsibility that varies by service model (IaaS, PaaS, SaaS). In SaaS models, this responsibility lies with the provider. In PaaS models, it is shared. In IaaS, it is the customer's responsibility. Customers can secure data at rest through encryption with key management services, access controls, and regular backups. Most managed database services offer encryption by default.

Data in Transit, meaning data traveling between services or across the cloud networking boundary, is also the responsibility of the cloud customer. Customers can use Secure Sockets Layer/Transport Layer Security (SSL/TLS), an encryption protocol that secures communication between clients and servers. Cloud-based data storage services support SSL/TLS. Most cloud-based data storage services support TLS.

In **Data Poisoning**, an attacker alters a model's data to change its behavior or output. This can happen by adding bad data to, or removing good data from, a model's training data set. This can result in degraded performance or a "back door" to the model itself.

In **Evasion Attacks**, a model receives malicious inputs at inference time that are intended to affect its response. An attacker can make small changes to the input that circumvent the model's ability to classify, without needing knowledge of its architecture, weights, or biases.

Identity and Access Management (IAM) governs access to cloud resources. Customers define identities such as users, roles, and groups, and grant them permissions through policies, typically written as JSON documents. They can also create resource-based policies that specify which identities are allowed to perform particular actions on a given resource.

Infrastructure as a Service (IaaS) offers virtualized resources on demand, such as compute, storage, and networking. Cloud virtual machines are a common example. A cloud infrastructure provider offers servers with a choice of operating systems and a customizable software stack, much like a physical server would be configured. Users can start and stop servers, install software packages, attach virtual disks, and configure access permissions. *This term was first introduced in module 2.*

A **Key Management Service (KMS)** secures data at rest by encrypting it using customer-managed keys. These services allow customers to manage encryption keys used to protect data, either by using provider-managed keys or by creating and managing their own customer-managed keys.

Machine Learning Operations (MLOps) is the set of practices used to develop and maintain machine learning models and datasets as throughputs of software build systems. ML Ops exists within CI and CD, and shares the motivations with those practices.

In **Model Extraction**, a malicious actor attempts to re-create an AI platform's machine learning model, often by reverse engineering its architecture, weights, and biases. Attackers can query the target model and use the responses, or steal training data, to build an approximation. They can then query the stolen model for outputs that the original would not reveal. This can lead to significant losses of intellectual property, confidential data, competitive advantage, and revenue, since unauthorized parties can use the stolen model without a license.

Platform as a Service (PaaS) provides a higher level of abstraction that makes the cloud easily programmable. It offers developers an environment in which to create and deploy applications without managing underlying compute resources, such as CPU and RAM. PaaS platforms provide multiple programming models and specialized services as building blocks for applications, including data access, authentication, and payment processing. AWS Relational Database Service (RDS) is an example of PaaS, offering a managed database service that abstracts setup, patching, backups, and other maintenance tasks. *This term was first introduced in module 2.*

The **Principle of Least Privilege** guides cloud practitioners to grant users access only to the resources they need. This helps secure cloud environments by limiting exposure in the event of a compromised account. If a malicious actor attempts to harm an organization's cloud resources, their IAM policies restrict them to specific actions on specific resources.

Transport Layer Security (TLS) encrypts and secures network communication for use by web-based applications, including browsers. TLS use starts with a handshake between a client and server, followed by authentication via certificate, and exchange of session keys. The secure connection then begins, with session keys encrypting its traffic.

The **Shared Responsibility Model** defines the division of responsibility between cloud service providers and customers. Cloud service providers are responsible for security "of the cloud", meaning providers own and maintain physical hardware and virtualization layers of cloud infrastructure. Cloud service customers are responsible for security "in the cloud", meaning they own and maintain guest operating systems, installed software, data, and networking configurations. Providers expect customers to take advantage of tools made available for managing access to their resources.

Software as a Service (SaaS) offers the highest level of abstraction, delivering applications that reside on top of the cloud stack, accessible by end users through web portals. This offers consumers the ability to shift from locally-installed computer programs to online software services that offer the same functionality. Traditional desktop applications can now be accessed as a service on the web (or works as a wrapper for the web service). This removes the customer burden of software maintenance and simplifies development and testing for providers. Many Google Workspace offerings (Mail, Maps, Sheets, etc.) use the SaaS model. Website-based applications like Anthropic's Claude web portal also use the SaaS model. *This term was first introduced in module 2.*

Zero Trust Architecture is a security model formalized by the National Institute of Standards and Technology (NIST) in 2020. It centers on the principle of "never trust, always verify," and enforces strict access controls that give users only the minimum permissions needed to access only the resources they require. This approach minimizes the risk and impact of unauthorized access or compromise.

8.3 Introduction to Cloud Security for AI Systems

Cloud Security Fundamentals

As artificial intelligence becomes more integrated into our cloud infrastructure, we're seeing entirely new categories of threats emerge. This lesson explores three major attack vectors that every AI practitioner needs to understand: model stealing, data poisoning, and evasion attacks. We'll also examine how the shared responsibility model works in cloud environments, understanding exactly where your security obligations begin and where your cloud provider's responsibilities end.

These aren't just theoretical concepts—these are real threats happening right now in production AI systems.

The Shared Responsibility Model

We can describe the boundary of responsibility for cloud security between users and providers with the statement: the user manages **security in the cloud**, while the provider manages **security of the cloud**.

User Responsibilities

Among the different services and resources offered by a provider, the user must:

- Secure data, applications, and access
- Manage encryption
- Configure network settings
- Manage identities

Provider Responsibilities

The provider must:

- Secure the physical hardware and networking within data centers and regions
- Host managed services which remove some burdens from the user

For example, running Python scripts on a virtual machine service like AWS EC2 requires the installation, configuration, and management of an operating system and dependencies, while the same application could run in a serverless resource like AWS Lambda without the additional setup and maintenance burden.

Major AI Security Threats

Cloud users managing artificial intelligence models protect their systems by applying best practices to prevent attacks. There are two main threats—model stealing and data poisoning—each with the ability to compromise system integrity and severely damage a company's reputation and bottom line.

Model Stealing

In **model stealing**, a malicious actor tries recreating an AI platform's machine learning model, often through reverse engineering architecture, weights, and biases. Attackers can query the target model and use the responses, or steal training data to build an approximation.

They then query the stolen model for responses the original would not reveal.

Consequences

- Expensive loss of intellectual property
- Confidential data exposure
- Loss of competitive edge
- Revenue loss
- Unauthorized parties can use the stolen model without a license

How It Works

An attacker creates their own shadow model, training it through a process of querying a target model and then using its response as an input to the shadow.

Data Poisoning

In **data poisoning**, an attacker introduces malicious data into a model's training dataset. This corrupts it and causes it to learn biased or incorrect information so that when a user submits queries, the responses the users receive become lower in quality.

Attack Vectors

An attacker may introduce bad data:

- Directly
- Through insider or backdoor access
- Through the model supply chain, meaning its dependencies like third-party software libraries

Consequences

A compromised model may:

- Show lower reliability and accuracy
- Become more vulnerable to future attacks
- Share confidential information

How It Works

An attacker gains access to the training data, learns its structure, and then uses that information to add or substitute their own data, yielding different results at inference.

Evasion Attacks

Evasion attacks comprise another class of security risk for AI models. They resemble data poisoning attacks in that the model receives inputs altered in a way intended to affect its response, but this time at inference time, not during training.

A bad actor can make minor changes to a model's input that circumvent a model's ability to classify without needing to understand its architecture, weights, or biases. This can also lead to the exposure of confidential data.

Real-World Example

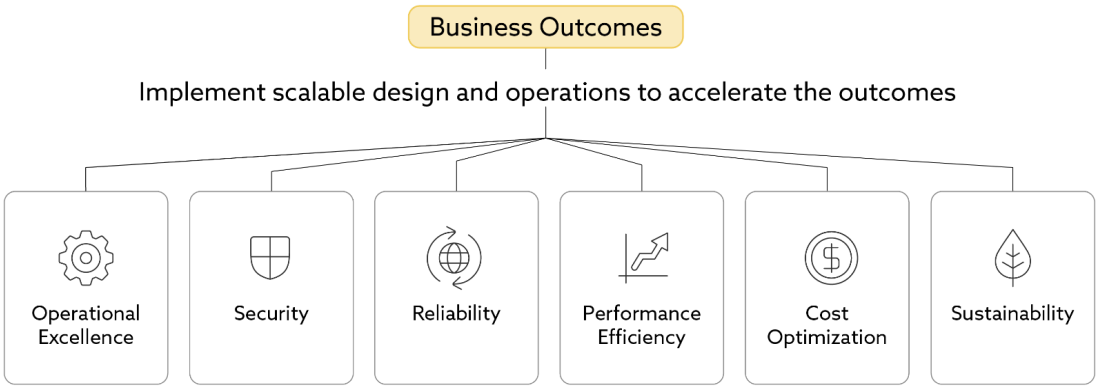
Researchers changed a model input and successfully deceived an autonomous driving system, indicating the potential for deadly consequences.

Comprehensive Security Strategy

At both training and inference stages, malicious actors can attack the system in a number of different ways. For developers and maintainers of cloud systems with machine learning components, knowing where and how these attacks work enables adoption of preventative and mitigating strategies. Combined with best practices for architecting cloud systems, practitioners have a comprehensive toolset to create and improve secure AI systems.

AWS Well-Architected Framework

The AWS Well-Architected Framework outlines sets of recommendations for making optimal architectural decisions in the cloud, organized by key areas of consideration (pillars) for well-designed cloud systems. Understanding these, and how to apply their concepts through patterns and services, allows cloud practitioners to create high-performing products at scale. For security of AI systems in the cloud, practitioners should pay attention to the security, reliability, and operational excellence pillars; they are particularly relevant for AI security, though all six pillars remain important for balanced architecture. These pillars guide design decisions for distinct objectives in a way that creates reinforcement and synergy between them.



<Alternative Text: The six pillars of the AWS Well-Architected framework are operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability.>

Security Pillar

The Security Pillar of the AWS Well-Architected Framework outlines how to protect data, systems, and processes from unwanted access and attacks. This includes risk assessment, managing access and permissions, detecting security events, and protecting systems and services. Its objective is to protect information, systems, and assets through assessment and mitigation, focusing on key areas such as identity and access management, detective controls, infrastructure protection, data protection, and incident response.

Design Principles for security include:

- **Strong identity foundation:** Following the principle of least privilege and enforcing separation of duties
- **Traceability:** Ensuring real-time logging, monitoring, metric collection, and alerting
- **Security at all layers:** Ensuring breaches of one level will not breach others
- **Automation of security best practices:** Reducing potential of human error
- **Protection of data in transit and at rest:** Using encryption, tokenization, and access control
- **Minimal data access:** Keep data exposure as small as possible by only giving access to what's absolutely needed
- **Preparation for security events:** Creating incident management and investigation policies, automating incident responses

Reliability Pillar

The Reliability Pillar of the AWS Well-Architected Framework outlines how to design and build for high availability and fault tolerance, ensuring that applications remain accessible when needed. Its objective is to prevent failures and recover quickly when they occur, focusing on key areas such as requirements, change management, and failure management. This involves designing systems to withstand or overcome adverse conditions, planning for recovery, and managing change effectively.

Design Principles for reliability include:

- **Testing recovery procedures:** Simulating failure scenarios on duplicated infrastructure
- **Automation of failure recovery:** Reducing downtime, impact on users, and need for manual intervention
- **Elastic handling of changing capacity needs:** Ensuring cloud resource supply meets system demand at all times
- **Design systems to scale out horizontally so they can handle failures better and stay available:** Adding resources as needed to replace failed ones
- **Automating change management:** Self-executing patches and deployment, reducing manual intervention, using automation and infrastructure as code to manage changes safely and consistently

Operational Excellence Pillar

The Operational Excellence Pillar of the AWS Well-Architected Framework focuses on ensuring that cloud architectures meet organizational objectives and goals. Its objective is to operate systems that deliver value while driving continual improvement of supporting processes and procedures, with key concepts of preparation, operation, and evolution. Operational excellence means designing solutions that address a problem effectively and continue to improve over time. This involves preparing and operating architectures to meet objectives, monitoring systems for improvement, managing changes effectively, and responding to events.

Design principles for operational excellence include:

- **Operations as code:** Version/revision control, loose coupling, scripted and/or automated procedures, and continuous integration/continuous delivery
- **Annotated documentation changes:** Organizing updates and keeping document versions accurate and up to date
- **Small reversible changes and frequent refinement:** Improvements made based on monitored metrics and other analysis with the ability to immediately roll back deployments
- **Anticipation of failure:** Elimination of single points of failure, testing failure scenarios, preparing and automating response and recovery
- **Learning from all operational events and failures:** Reflect on lessons learned from events and failures and share those learnings with your organization

Taken together, the Security, Reliability, and Operational Excellence pillars give cloud practitioners a roadmap for a system architecture with powerful diagnostics, built-in stability, and mechanisms for handling unexpected behaviors. The level of customer responsibility for security depends on the cloud service model. In SaaS, the provider manages everything except API keys, while in IaaS, customers are fully responsible for security in the cloud. In IaaS the full onus of security "in the cloud" lies with the customer, including encryption, keys, identity, network controls, and application configuration. In SaaS the provider manages everything except identity and access management (often just API keys). In PaaS the degree of customer responsibility lies somewhere between IaaS and Saas, and depends on the service(s) used.

8.4 Secure Deployment and Management of AI Models on Cloud Platforms

Identity and Access Management (IAM)

Identity and Access Management (IAM) governs access to cloud resources. IAM identities (users, groups, roles) are granted permissions through policies. Administrators attach policies to identities or resources to control actions on resources.

Practitioners should keep policies as narrow and granular as possible, following the principle of least privilege to limit the impacts of a potential breach. The principle of least privilege states that users should only be granted the minimum level of access needed to complete their authorized tasks. For example, an account administrator may grant one group of QA users on a machine learning model permissions to perform inference, and grant another group of data curation users permissions to update a database or data store. Depending on the service and provider, database access may be controlled at the table level or database level. IAM policies should be periodically reviewed to ensure they remain effective and up to date.

Key Management Service (KMS)

A Key Management Service (KMS) offers a centralized platform for managing encryption keys. Many cloud services utilize KMS to automatically encrypt data at rest, while applications can use it for a process called envelope encryption. By default, services often use keys owned and managed by the cloud provider, but users also have the option to create and control their own customer-managed keys (CMKs), which allow for customized rotation schedules and access policies.. If an attacker gained access to sensitive data at rest, they cannot use it without first decrypting it, for which they would need access to a key via the KMS. In the event of a compromised key, an administrator can rotate the key, re-encrypt data with the new key, and change IAM policies to revoke access from the offending user.

In AI deployments, KMS takes on a more critical role, not only securing the massive, often sensitive, training datasets but also encrypting the highly valuable and proprietary AI models themselves. The trained model represents significant intellectual property and business value, making its security a top priority. The keys managed by KMS also provide granular access control, ensuring that only authorized services and users can interact with different stages of the AI pipeline, from training to inference.

Virtual Private Cloud (VPC)

A virtual private cloud (VPC) is logically isolated within a public cloud provider's infrastructure, allowing users to define their own virtual network topology while abstracting away the physical infrastructure. Virtual Private Clouds allow for isolating data and traffic, and controlling access through configuration of virtual networks, firewalls, access control lists, and security groups. In a typical cloud deployment, a VPC serves as a fundamental security layer, isolating a company's resources in a private network to control traffic and prevent unauthorized external access. For AI deployments, the VPC is used to create a highly secure and isolated environment for handling sensitive data. It ensures that large-scale training data and the resulting models never traverse the public internet, a critical requirement for data privacy and regulatory compliance. Furthermore, a VPC allows for the deployment of private API endpoints for model

inference, ensuring that the AI service is only accessible from within the organization's network, thereby protecting it from external threats and unauthorized access.

Infrastructure as Code (IaC)

Infrastructure as code (IaC) enables creation and management of cloud resources through declarative configuration files. These resources include those mentioned above, and others used in cloud-based ML applications. These files are human-readable, machine-consumable descriptions of architectures as sets of interacting resources. This allows practitioners to perform operations as code, experiment safely, replicate environments, practice failures, and more. It also facilitates automation, and can be used for disaster and resource recovery, and for setting up test environments.

IaC provides a powerful, efficient alternative to manually provisioning and configuring infrastructure. Cloud practitioners can use one template to declare a desired state for multiple similar environments. They can deploy and manage configuration drift rapidly, using either built-in or third-party drift-detection features. All of this reduces the potential for human error, saving time and effort.

IaC is a standard practice for automating the deployment of consistent and reproducible infrastructure in common cloud scenarios. Its application in AI deployments is far more specialized and complex. IaC is used to define and manage highly specific, large-scale resources essential for AI workloads. This includes setting up GPU clusters, configuring specialized networking for distributed training, and automating the deployment of container orchestration platforms. The use of IaC in AI is crucial for iterative model training and scaling, as it allows data scientists and MLOps engineers to rapidly and consistently spin up and tear down complex, resource-intensive environments on demand.

8.5 Continuous Monitoring and MLOps Pipeline Security

Deployment and Management

Picture this: your AI model is performing perfectly in testing, but three months after deployment, it starts making dangerous misclassifications. Was it a gradual drift, or did someone poison your training pipeline six months ago? Without the right security framework, you might never know. This is exactly why understanding threats isn't enough. We need to build AI systems that can actually defend themselves.

This lesson explores the operational security of AI systems, covering the six core components of cloud security, from data protection and compliance to disaster recovery planning. The goal is to provide a clear roadmap for deploying AI models securely and maintaining their security throughout their lifecycle.

Shared Responsibility Model

When building applications for the cloud, the user manages **security in the cloud** while the provider manages **security of the cloud**. Users need to understand points of failure and best practices for architecture, specifically with respect to security, reliability, and operational excellence.

Familiarity with strategies for securing data, ensuring service availability, adherence to compliance and governance requirements, disaster recovery planning, and identity and access management all help with prevention and mitigation of threats.

Logging and Monitoring

Logging and monitoring comprise critical operational features in cloud applications, granting the organization insights into whether its product meets its needs and goals, how efficiently it does so, and how to make incremental improvements.

Services like AWS CloudWatch show not only system-critical metrics like availability but also detailed logs, which practitioners can organize into custom metrics. By default, artificial intelligence systems lack observability into a lot of important data and information, which the team should work together to make accessible.

By gathering these data into metrics, the team further enables automation—for example, automatically scaling resources during periods of busy use or sending alerts on Slack to investigate anomalous behaviors.

Logging

Logging is a critical diagnostic tool for software development, and a straightforward way to make foundational data available to an organization. Not only can developers use log data to triage and fix defects, it can help security teams perform audits, and legal teams ensure compliance. For ease of use with querying tools, developers should organize logs into a standard format. Developers should also use severity levels to highlight issues requiring attention, including automated alerts and responses. Detailed logging of data access, resource use, and API calls can be particularly useful for understanding and responding to security incidents.

Cloud-based AI applications bring security challenges unique to non-AI applications, for which logging is a primary diagnostic tool. Bad actors can try to exploit inference endpoints with evasion and model-stealing attacks. When models process sensitive data the need to ensure privacy becomes acute, as the organization's measures to prevent and mitigate leaking of such data become essential to the application. Detailed logging of inference API calls is important to catch unusual query behavior that might point to adversarial probing or model extraction. At the same time, logs should be designed with care so they don't accidentally reveal sensitive training data or model outputs. AI systems also need diagnostics for model behavior, like confidence scores, distributions, and drift detection. The organization needs information about both system infrastructure and model performance, as both can show information about attacks.

Monitoring

Monitoring is a critical operational feature in cloud applications, providing organizations with insights into whether their products meet business needs and goals, how efficiently they operate, and where improvements can be made. It supports efficient operations at scale by maximizing agility, minimizing downtime, enabling experimentation and testing, managing risk, and responding to threats or failures. Cloud provider monitoring services provide real-time insights into service and infrastructure health and performance, while also allowing users to define custom metrics and create data visualizations for non-development team members. These services often include features for filtering log events and generating custom metrics and monitoring from selected log data. For AI models, alerts shouldn't just fire on system resource thresholds. They should also detect abnormal usage patterns like unexpected surges in inference calls or suspicious query behavior, so teams can respond quickly to potential misuse or attacks.

For interoperable distributed tracing, implement the W3C Trace Context standard using traceparent and tracestate headers. When working within AWS, you can achieve this by instrumenting your applications with either the AWS X-Ray SDK or an OpenTelemetry exporter that sends data to AWS X-Ray. This can help with understanding and diagnosing system behaviors in distributed applications. For example, an application may route user requests through a DNS service, then an API gateway, followed by a serverless function that invokes an inference endpoint. This path involves at least four services, so identifying and observing user behavior would otherwise be difficult. Traceability is essential for real-time monitoring of activity and changes in a cloud application, so that when something does go wrong, the organization can perform root cause analysis, assess impact, and learn from the incident.

Alerting

By aggregating these data into metrics, the team enables further automation, such as automatically scaling resources during peak usage or sending Slack alerts to investigate anomalous behavior.

In AWS, CloudWatch offers monitoring through metrics, logs, dashboards, and alarms, while CloudTrail logs account activity and API calls for auditing. For performance analysis, AWS X-Ray provides distributed tracing, and EventBridge manages event routing to automate workflows, while CloudWatch shows metrics like availability and resource usage, and detailed logs, which practitioners can organize into custom metrics. CloudWatch metrics can serve as inputs to automated behaviors, such as alerts when resources exceed some defined thresholds, for example incoming requests per second. A user starting with good logging practices in their code can use CloudWatch to observe their logs in real-time, create custom metrics and monitoring, and use those to create alerts. For AI models, alerts should also flag unusual activity such as sudden surges in queries that could indicate someone is attempting to scrape or copy the model, or repeated odd inputs that might signal adversarial testing.

<Alternative Text: CloudWatch provides many services, including logs, metrics, alarms, X-Ray, agent, events, and dashboards.>

Supply Chain

Through applications of good practices with respect to IAM and KMS, practitioners can secure an AI model's data. However, there exist other dependencies for the model's code itself, requiring a different set of strategies. Any code depending on third-party libraries or modules also requires tooling for installing and managing those dependencies, which developers can use to prevent and address problems, including security issues. Specifying, or "pinning" dependencies facilitates consistency and reproducibility of software across environments. Further, using lockfiles with checksums guarantees the versions of those dependencies by enforcing data integrity checks.

Any breaking changes, including vulnerabilities, would not be automatically included in the software. Coupled with automated security scans for Common Vulnerabilities and Exposures (CVE), developers can observe and control their software supply chain and protect it against undesired external effects. Model artifacts are valuable assets the organization should verify, version, and limit access to. In an insecure ML Ops system, bad actors can poison training data to corrupt models, or insert attack vectors in software dependencies. For more information about ML Ops, see this comprehensive CSA overview.

Continuous Development and Deployment

As developers make changes to models, they need a simple and straightforward way to ensure their work propagates through to a production environment. This pipeline for continuous development and deployment includes:

- Exporting the model into some usable format
- Tracking its metadata
- Running and observing the model and its relevant metrics
- Version control

Through automation, this allows for seamless integration of new changes, diagnostics for investigation and triaging, and reversion of these changes when needed.

Secure Deployment Infrastructure

The work required for secure deployment of AI models spans multiple sets of skills and responsibilities, which an organization may separate into individual teams.

Cloud Practitioner Responsibilities

Infrastructure Management: Cloud practitioners create and maintain system infrastructure and resources related to architecture, networking, and encryption, using infrastructure as code tools such as Terraform.

Identity and Access Management: They manage authentication and authorization to the system and its resources through identity and access management, granting only the minimum access required to the minimum number of users, roles, and/or identities.

Network Security: Using virtual private clouds and security groups adds another layer of security and isolation at the network level by specifying permissions for inter-service communication.

Encryption: Cloud APIs encrypt data in transit through HTTPS. A key management service adds encryption of data at rest in databases, object stores, and container registries. Keys can also encrypt the volumes of cloud AI resources, such as processing jobs, notebooks, training jobs, and model endpoints.

Software Supply Chain: The team must also manage the software supply chain, third-party libraries, and dependencies by versioning lock files and data integrity checks.

Deployment Models

This outlines secure deployment and management of AI models for PaaS and IaaS. For SaaS like cloud websites, account administrators manage authorization and authentication through permissions and API keys, and the cloud teams manage the underlying platform and infrastructure.

Incident Response and Disaster Recovery

In an ideal world, cloud systems would exhibit perfect fault tolerance and no downtime. But as Amazon CTO Werner Vogels said, "Everything fails all the time." Incidents happen, so teams must prepare themselves to mitigate their impact.

Incident Response

Incident response can be facilitated by:

- Detailed logging and monitoring of metrics like data access, resource use, and service calls
- Events set up to trigger automated responses
- Clean room environments to carry out forensics in a safe and isolated way

Disaster Recovery Planning

The team creates a plan that includes:

Define Objectives: Establish metrics for downtime and data loss

Organize People: Involve operations, testing, development, security, and stakeholders, ensuring everyone understands what they will do in the scenario

Restore Operations: Use data backups and create or modify any necessary resources, identifying and performing incident-related testing and maintenance

Teams can also organize **game days** to test their plan in anticipation of an actual problem in a simulated environment. It is possible to create these on-demand using infrastructure as code. Automating disaster

recovery processes also minimizes impact to users and the organization by reducing system downtime and time to recovery.

Zero Trust Architecture

Zero trust architecture is a security model created by the National Institute of Standards and Technology in 2020, defined in terms of seven principles:

1. All data sources and computing services are considered resources
2. All communication is secured regardless of network location
3. Access to individual enterprise resources is granted on a per-session basis
4. Access to resources is determined by dynamic policies, including the observable state of client identity, application or service, and the requesting asset, and may include other behavioral and environmental attributes
5. The enterprise monitors and measures the integrity and security posture of all owned and associated assets
6. All resource authentication and authorization are dynamic and strictly enforced before access is allowed
7. The enterprise collects as much information as possible about the current state of assets, network infrastructure, and communications and uses it to improve its security posture

A zero trust system implements a comprehensive set of policies that give a user the minimum required permissions to the fewest required resources to reduce the impact of unauthorized access as much as possible.

Key Takeaways

AI security is a continuous process, not a one-time setup. The monitoring strategies discussed—tracking data health, model performance, and service availability—aren't just nice-to-haves; they're essential for catching attacks and drift before they impact your business.

Zero trust architecture provides the foundation, but it's the MLOps pipeline security and continuous monitoring that will keep your system safe day after day. As AI systems become more critical to business operations, these security practices will only become more important. Remember, the threat landscape is always evolving.

8.6 Incident Response and Disaster Recovery for AI in the Cloud

Murphy's Law

Murphy's Law, a well-known engineering principle, states that "anything that can go wrong will go wrong." AWS CTO Werner Vogels often emphasizes an even less optimistic version: "Everything fails all the time." For cloud practitioners, acceptance of failure as predictable and inevitable encourages architecture and design with security and resilience in mind. By using best practices like logging, monitoring, and IaC, account for failure by incorporating response and recovery into systems.

Incident Response

Responses to incidents can be facilitated by detailed logging of data access, resource use, API calls, trace headers, events set up to trigger automated responses, and using pre-established "clean room" environments to carry out forensics in a safe and isolated way. Teams should use runbooks, a detailed set of pre-planned steps to take during an incident, for standard activities. When the team identifies a threat, it should use available diagnostic tools to understand the problem and find a root cause. The team should then isolate and remove the problem and restore system functionality. For an AI system, they may do things like quarantine or disable an inference endpoint, roll back a model version, or revoke tool connector credentials.

The team should regularly review its incident response processes, including after execution. This will help the team continually improve its systems and processes. Some examples of improvements may include:

- Integrating functional testing as part of deployment; allowing rollback of deployments if tests fail
- Integrating resilience tests as part of deployment
- Deploying using immutable (replacing instead of changing) infrastructure
- Deploying changes with automation

Disaster Recovery

When system components fail, efficient recovery depends on the team following a plan. Ideally this plan exists beforehand, but this may not always be the case. The team should start by defining recovery objectives for downtime and data loss i.e. Recovery Time Objective (RTO) and Recovery Point Objective (RPO). In general, there is an inverse correlation between these metrics and cost; the lower the desired RTO and RPO, the higher the cost to the organization. The team should use its defined recovery strategies to meet these recovery objectives, and then validate the disaster recovery execution with thorough testing. The team should then manage configuration drift (slight changes that accumulate over time) at the disaster recovery site or region. Finally, the team should learn from the disaster, becoming more efficient at its responses. This includes automating recovery and minimizing manual intervention, facilitated by continuous monitoring and automated testing.
Business Continuity is the organization's capability to maintain or quickly resume delivery of critical services and operations during, and after, a disruption. It is achieved through a structured program of

policies, procedures, and strategies that ensure operational resilience, minimize downtime, and reduce risk exposure.

An effective Business Continuity Plan involves identifying critical assets and interdependencies, assessing risks and impacts, setting recovery objectives (RTO and RPO), and defining strategies to maintain or restore operations. The outcome is improved resilience, reduced downtime, and greater confidence that the organization can continue operating despite disruptions. Unlike standard infrastructure, AI incidents may require retraining from validated checkpoints or reverting to prior model versions in addition to infrastructure recovery.

Game Days

Teams can also conduct "game days" to simulate failures and exercise response procedures, sometimes even in production environments, provided risks are well-managed and customer impact is minimized.

To prepare for a game day, a team first selects workloads and scenarios. The team then organizes necessary personnel into roles and responsibilities: operations, testing, development, security, business and stakeholders. They then define the simulated events to test, such as previous failures, known vulnerabilities, weaknesses, or projected limits and tolerances. They treat the simulations as if they were real events, following all relevant defined processes.The team then builds the environment using IaC templates and IAM to isolate the game day environment. Monitoring must exist as in production.

To execute the game day simulations, the team runs the exercise(s) for the required amount of time. The failure/test scenarios must happen with the correct sequence and timing to represent reality. Team monitors will observe behavior of the people involved with the process. The exercise(s) end upon reaching their objective(s). The team preserves logs for analysis, and removes the test environment.

The team then analyzes the results and improves. First, it holds a debrief and identifies areas where processes and procedures need to be improved. This includes reviewing logs to ensure that monitoring and alerting processes work as expected. The team may identify opportunities for automation, additional training, education, and additional areas for subsequent game days and areas of testing. Finally, it should improve in those areas where identified issues exist.

You can optionally read more about game days in the AWS Well-Architected Framework Documentation.

8.7 Zero Trust Architecture for AI Workloads and Cloud Deployments

Zero Trust Architecture

Zero Trust Architecture is a security model formalized by the National Institute of Standards and Technology in 2020, defined in terms of seven principles:

1. All data sources and computing services are considered resources
2. All communication is secured regardless of network location
3. Access to individual enterprise resources is granted on a per-session basis
4. Access to resources is determined by dynamic policy—including the observable state of client identity, application/service, and the requesting asset—and may include other behavioral and environmental attributes
5. The enterprise monitors and measures the integrity and security posture of all owned and associated assets
6. All resource authentication and authorization are dynamic and strictly enforced before access is allowed
7. The enterprise collects as much information as possible about the current state of assets, network infrastructure and communications and uses it to improve its security posture

A zero trust system is a comprehensive set of policies that gives a user the minimum required permissions to the fewest required resources, to reduce the impact of unauthorized access as much as possible. Its guiding principle is “never trust, always verify”. By minimizing access, a zero trust system also minimizes both exposure to and impact from security incidents.

Applying zero trust principles addresses unique security concerns for cloud-based AI systems. ML models and data are first-class resources needing precise and least-privileged access controls, distinguished by entity between training, fine-tuning, and inference. Dynamic policies evaluate user identity and data sensitivity levels, model capabilities, and misuse (e.g. rate-limiting an inference API). Model outputs require continuous monitoring for hallucinations, leakage, and attacks. Monitoring also includes model performance metrics like confidence thresholds, distributions, and anomaly detection, which may show extraction or poisoning attacks. Model signing, secure storage, and runtime verification ensure system integrity, and audit trails track changes over time, like additions to training data and parameter tuning.

Similarly, practitioners should treat AI agents as user-like entities. Every agent action should require real-time verification regardless of previous authentication. Systems should expect that agents may be compromised or manipulated. Agents should receive only the minimum access required for their specific task. The implementation strategy follows. The system should continuously verify agent identity and

behavior through behavioral analysis and anomaly detection. The system should micro-segment AI environments to limit lateral movement if agents are compromised. It should also include dynamic policy enforcement that adapts to real-time risk assessments.

Optional Continued Reading

Click here to download a section from the NIST publication "Zero Trust Architecture". This reading will cover four deployment scenarios/use cases of the Zero Trust Architecture.

Additionally, see the AWS or Google Cloud documentation for guidance on how to build a zero trust architecture.

8.8 Summary

This module equipped you with essential knowledge for securing AI systems in cloud environments, covering everything from foundational security principles to advanced Zero Trust implementations. You learned how to apply the AWS Well-Architected Framework's security, reliability, and operational excellence pillars specifically to AI workloads, while mastering practical skills in secure deployment strategies, continuous monitoring, and incident response planning. The module emphasized the shared responsibility model and how security considerations shift across different cloud service models (SaaS, PaaS, IaaS) when deploying AI systems.

Key Concepts Mastered

Identity and Access Management (IAM) and Encryption

You gained insight into implementing granular access controls following the principle of least privilege, managing user roles and groups through JSON-based policies, and utilizing temporary security credentials via security token services. You learned to secure data at rest using Key Management Services (KMS) with customer-managed encryption keys, implement automatic key rotation strategies, and protect data in transit using TLS while understanding the shared responsibility model for data protection.

MLOps Pipeline Security and Continuous Monitoring

You learned about establishing secure MLOps workflows through comprehensive logging, real-time monitoring, and automated alerting systems. You learned to implement trace headers for distributed AI applications, create custom metrics and dashboards, establish automated scaling and incident response triggers, and secure the software supply chain through dependency pinning, lockfiles, and automated vulnerability scanning for Common Vulnerabilities and Exposures (CVE).

Zero Trust Architecture and Incident Response

You explored NIST's Zero Trust Architecture, including the implementation of "never trust, always verify" policies that grant minimum required permissions to the fewest required resources. You developed skills in creating comprehensive incident response plans using runbooks, conducting "game days" to simulate failures and test response procedures, implementing disaster recovery strategies with defined Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO), and using Infrastructure as Code (IaC) for consistent, automated recovery processes.

Looking Ahead: Your Continued AI Journey

The cloud security topics covered in this module are fundamental for any AI practitioner working in modern enterprise environments, where the majority of AI systems are deployed on cloud platforms. As AI systems become more integrated into critical business processes, your ability to implement Zero Trust principles, design secure MLOps pipelines, and respond effectively to incidents will be invaluable.

Moving forward, consider deepening your expertise in specific cloud provider certifications (AWS, Azure, GCP), staying current with emerging AI security threats like model stealing and evasion attacks, and

contributing to your organization's AI governance frameworks. The incident response and disaster recovery skills you have developed will prove essential as AI systems scale and become mission-critical. Remember that security is not a one-time implementation but an ongoing practice requiring continuous monitoring, regular testing through game days, and adaptation to new threats in the rapidly evolving AI landscape.

Change Log

Version #	Approved by	Notes
20251015	HMR	Initial Release